Abnormal Logical Representation Learning for Intrusion Detection in Industrial Control Systems

Xinyu Xu, Yingxu Lai, *Member, IEEE*, Xiao Zhang, and Xinrui Dong

***Abstract*—As security threats to industrial control systems become more prevalent, it is imperative to deploy effective intrusion-detection systems. However, the existing methods are insufficient for addressing contemporary attacks. Rule-based methods are heavily dependent on manual settings, and the covertness of attacks poses challenges to rule effectiveness. Machine and deep learning methods exhibit low interpretability owing to their complex designs, and the semantic gap between the model and the actual operational interpretation limits their applicability. To mitigate these shortcomings, we propose an abnormal logical representation learning (ALRL) intrusion detection method for industrial control systems. ALRL contains a specific lightweight neural network and employs knowledge distillation to achieve high classification ability. More importantly, it can generate effective and concise intrusion detection rules directly from the learned knowledge of the model. The hierarchical model structure and residual connections ensure high interpretability of the rules. Experiments conducted on two publicly available industrial control datasets demonstrate that ALRL can classify attacks with an excellent performance. In addition, the logical rules generated by ALRL can effectively detect all types of attacks and exhibit good interpretability.**

***Index Terms*—Control logic, industrial control system (ICS), intrusion detection, model interpretability, rule generation.**

I. Introduction

A. Background and Motivation

INDUSTRIAL control systems (ICSs) are widely used to control and monitor physical processes closely related to critical infrastructure, playing a crucial role in maintaining a nation's stable development [1]. However, ICSs have transitioned from their previous isolated to interconnected owing to the rapid development of networks, leading to a growing number of security threats. For example, in January 2022, Oiltanking, a major fuel distribution company in Germany, suffered a cyberattack, which resulted in the disruption of supply to nearly 200 fuel stations [2]. This incident highlighted the significant threat posed by cybersecurity attacks on ICSs, underscoring the crucial role of practical and effective intrusion-detection systems (IDSs) in safeguarding ICSs.

B. Limitations of Prior Methods

Currently, most ICSs employ rule-based methods for IDSs, which are recognized for their reliability and human-readable representation of attack behaviors [3], [4], [5]. The advantage of rule-based methods is that ICSs can execute actions consistently and deterministically because rules define the system's behavior and decision logic. Unfortunately, existing rule-based methods fail to satisfy the requirements for detecting critical attacks in ICSs, primarily for two key reasons. First, the creation of detection rules necessitates the involvement of domain experts throughout the process, making it time-consuming and labor-intensive to replicate the procedure for each distinct ICS. Second, attackers often employ diverse strategies to conceal their activities, making it challenging for rules solely rooted in expert knowledge to comprehensively encompass the essential elements of these attacks. Therefore, rule-based methods can have high false-positive rates and suffer from low detection rates in realistic attack scenarios, which can lead to significant damage to physical processes.

In contrast, machine learning (ML)- and deep learning (DL)-based intrusion detection methods exhibit outstanding detection performance [6], [7], [8], [9], [10], [11]. However, deploying these methods directly in an ICS is challenging. Owing to the extensive volume of network traffic, misclassification generated by ML and DL methods requires security analysts to spend a substantial amount of time and effort manually reviewing false system alarms, resulting in high error costs. This drawback is rooted in the sophisticated design of the models intended to capture complex feature interactions, which suffer from low interpretability and transparency. A semantic gap exists between the model and the actual operational interpretation of administrators, making it challenging to translate detection results into

Manuscript received 7 November 2023; revised 13 March 2024 and 8 April 2024; accepted 17 April 2024. Date of publication 15 May 2024; date of current version 5 August 2024. This work was supported by the National Natural Science Foundation of China under Grant 63272017. Paper no. TII-23-4401. *(Corresponding author: Yingxu Lai.)*

Xinyu Xu, Xiao Zhang, and Xinrui Dong are with the Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China (e-mail: xuxy22@emails.bjut.edu.cn; xiao_coraline@emails.bjut.edu.cn; xinruidong@emails.bjut.edu.cn).

Yingxu Lai is with the Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China, and also with the Engineering Research Center of Intelligent Perception and Autonomous Control, Ministry of Education, Beijing 100124, China (e-mail: laiyingxu@bjut.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2024.3396348.

Digital Object Identifier 10.1109/TII.2024.3396348

1551-3203 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

actionable reports, such as deterministic rules [12]. Although some feature-based methods have been developed to explain detection results [13], [14], imprecise feature extraction and the complexity of intrusion alert descriptions may enable adversaries to hide malicious content, rendering administrators unable to grasp the underlying reasons behind attacks. Furthermore, feature-level explanations cannot reflect how and why detection models generate certain results because the corresponding models themselves are unexplainable.

C. Proposed Approach and Key Contributions

In this article, we propose an abnormal logical representation learning (ALRL) intrusion detection method for ICSs. Our method addresses two limitations of existing mainstream IDSs, including the limited quality of rule-based methods and the high training cost and low interpretability of DL methods. Specifically, we designed a lightweight neural network to capture the logical relationships between industrial devices and employed knowledge distillation to transfer knowledge from other models, enhancing the performance of attack classification. To ensure high interpretability, we represented the learned knowledge in the form of abnormal control logic rules based on logical neural-symbolic representation. For an intuitive understanding of the rules, we partitioned the model structure into distinct learning layers, showcasing the relationships among devices within and across physical stages. Through residual connections, we enable the model to adjust the complexity of rules based on different attack scenarios. The key contributions of this study are summarized as follows.

1) We propose ALRL, an intrusion detection method capable of accurately detecting attacks and generating deterministic and interpretable rules. These rules contain clear and precise attack information, bridging the gap between effective but opaque models and transparent but low-quality rules. Our method can help alleviate the workload of industrial maintainers in analyzing attack patterns and enhance the security of ICSs.
2) We designed a specific neural network to achieve precise identification of various attack behaviors in limited computational scenarios. A significant contribution of our work is the ability to directly generate effective and concise intrusion detection rules tailored for ICSs by utilizing model knowledge learned from the classification process. The hierarchical structure design based on physical stages and residual connections empowers the rules with explanatory capabilities.
3) We conducted extensive experiments on two publicly available industrial control datasets. The results demonstrate the effectiveness of ALRL in attack classification. Compared to other rule-based methods, ALRL successfully reduces the average length of extracted abnormal control logic rules by approximately 44.05% and 16.42% on the two datasets while maintaining similar attack detection performance. These rules exhibit outstanding attack discrimination and good interpretability.

II. Related Work

A. Rule-Based IDSs

Currently, rule-based IDSs are practical and prevalent within ICSs owing to their deployment flexibility and human-readable nature. These rules enable administrators to monitor the real-time status of the system conveniently. Adepu et al. [4] introduced a distributed method in form of rules for attack detection, which relies on expert knowledge of process dynamics and the entanglement of states among physical components. However, the intricate operational logic of contemporary ICSs increases the risk of overlooking the crucial details of abnormal behavior when relying solely on expert knowledge. Das et al. [5] conducted logical data analysis and designed a specific Boolean function for supervising attack classification. This approach learns the detection rules from historical sensor data and demonstrates excellent localization capabilities. Nevertheless, it overlooks the relationship between the actuators and sensors, thereby affecting its detection performance, especially in highly concealed attack scenarios. Ahmim et al. [6] proposed a hierarchical IDS based on decision trees and rule-based concepts, which exhibited high true-negative and detection rates. Although decision trees are highly interpretable models capable of directly extracting decision rules, balancing the precision and complexity of tree structures is challenging. The pursuit of high accuracy can result in overly complex models that generate excessive rule conditions.

B. ML- and DL-Based IDSs

Many existing approaches have incorporated ML techniques with intrusion detection and have exhibited outstanding detection performance. Zhang et al. [7] leveraged methods such as k-nearest neighbors and random forests to propose a multilayer IDS tailored for ICSs. This approach considers multiple data sources and effectively differentiates between normal and attack samples. Le et al. [8] proposed extreme gradient boosting (XGBoost) model to improve the detection of imbalanced distribution attack types in the industrial internet. Fang et al. [9] proposed an ICS feature selection method based on genetic algorithms, which involves feature ranking to eliminate redundant features and utilizes tree-growing clustering to enhance optimization speed.

DL methods are becoming increasingly prevalent, leading to improved detection accuracy. This is attributed to the complex model structures designed to capture feature relationships. Kravchikd et al. [10] introduced a one-dimensional convolutional neural network with autoencoders to detect anomalies in IDSs. This approach considers both the time and frequency domains of the data and demonstrates the effectiveness of principal component analysis for data preprocessing. Huong et al. [11] proposed a lightweight and robust distributed anomaly detection architecture, with the hybrid design of autoencoder, transformer, and fourier mixing sublayer.

However, DL methods often struggle to provide concrete evidence for model behavioral decisions, introducing uncertainty into predictive outcomes. To explain the decisions made

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

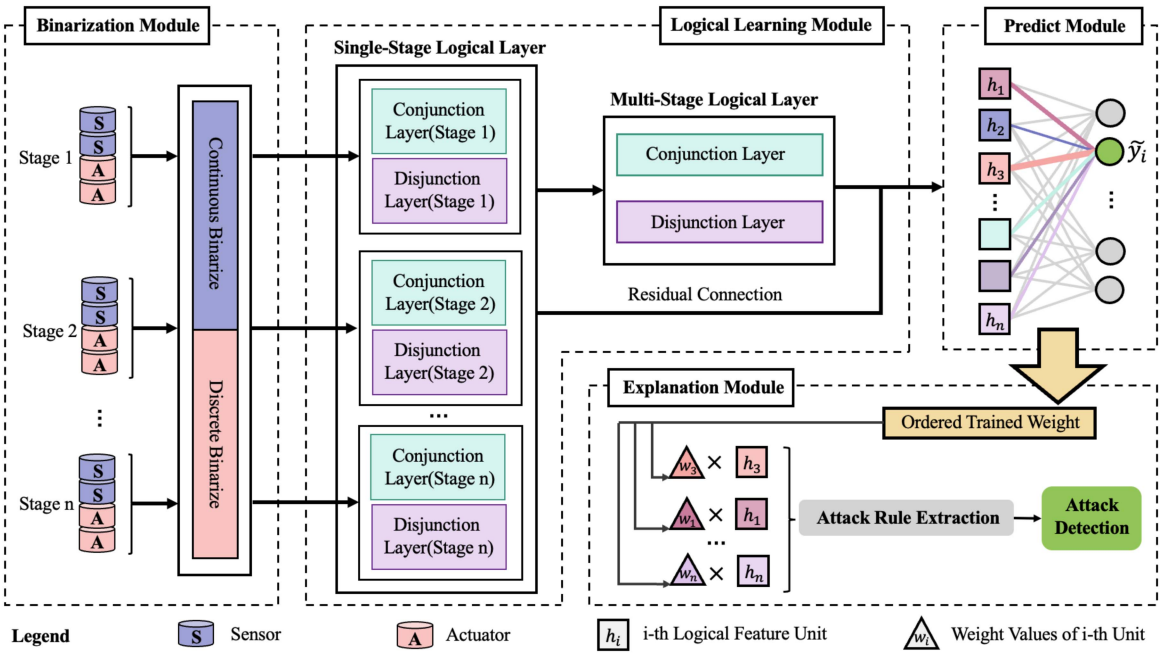


Fig. 1. Overview of the structure of ALRL.

TABLE I
COMPARISON OF EXISTING IDSS IN ICSS WITH ALRL†

Methods	Effectiveness	Lightweight	Interpretability	Rule Simplicity
Rule-based [4], [5], [6]	○	●	●	◐
ML-based [7], [8], [9]	◐	●	◐	/
DL-based [10], [11]				/
LU-IDS [13]	●	○	◐	◐
DL+SHAP [14]	●	○	◐	/
ALRL	●	●	●	●

† ●means strong, ◐means medium, ○means weak, while / means not support.

by the model, Sun et al. [13] utilized class-activation mapping to extract highly correlated attack feature points from a DL model. These points are employed to formulate rules based on historical information. However, the multitude of extracted features hinders administrators from gaining a comprehensive understanding of the state of the system. Bahadoripour et al. [14] employed Shapley additive explanations (SHAP) to identify how features contribute to attacks predicted by the model. Although the features reflect the attack characteristics to a certain extent, the model decision process remains opaque.

C. Comparison to Existing Research

We summarize the existing IDSs of ICSs in Table I. In terms of effectiveness, it denotes the model's capability to accurately identify attacks. In terms of lightweight, it signifies the number of trainable parameters in the model, assessing the required computational resources. ALRL combines the advantages of the two existing mainstream methods, achieving enhanced performance associated with the ML and DL methods while providing highly interpretable rules similar to rule-based methods. Although some methods are capable of generating rules, the conditions composing these rules are often overly complex, making their results uninterpretable. Conversely, our generated rules are sufficiently concise, enabling precise identification of attacks with a small number of rules and conditions, which can guide administrators to address anomalies promptly.

III. ABNORMAL LOGICAL REPRESENTATION LEARNING

A. Overview of the ALRL

An ALRL is a hierarchical IDS, as shown in Fig. 1. It comprises of four modules.

1) The binarization module is capable of binary encoding all sensors and actuators at each time step and stage.
2) The logical learning module employs a neural network to learn the logical relationships that exist within and across stages among the sensors and actuators.
3) The prediction module is responsible for performing attack classification tasks by utilizing the new features acquired from the preceding modules.
4) The explanation module is designed to extract knowledge from a trained network in the form of deterministic rules.

The following sections provide a detailed description of each system module.

B. Binarization Module

The binarization module is primarily responsible for converting the original feature values into one-dimensional tensors consisting of multiple zeros and ones. The outputs of the binarization layer correspond to the fundamental conditions that form the rules. To enhance the alignment of conditions within the ICS rules, we formulate specific criteria for different value categories.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

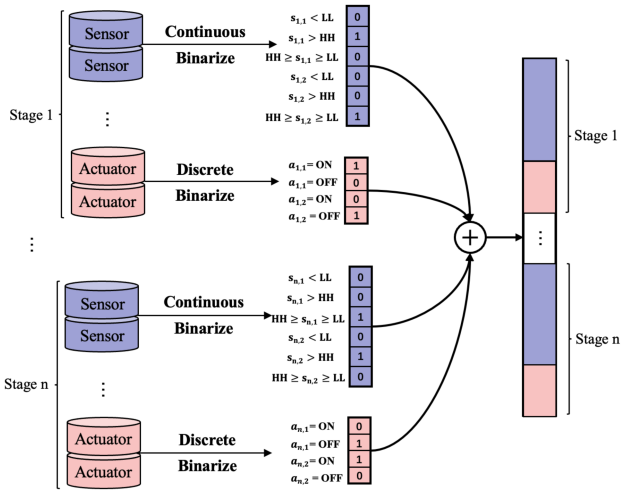


Fig. 2. Structure of the binarization module.

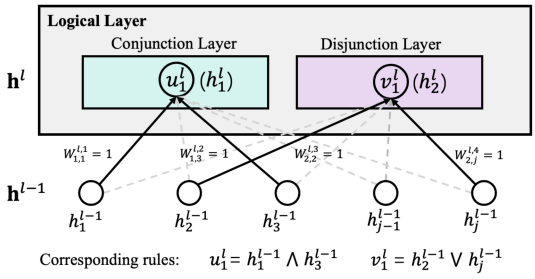


Fig. 3. Illustration of a logical representation.

As illustrated in Fig. 2, we utilized a continuous binarization layer to process sensor values, which refer to continuous data displayed by factory instruments highly relevant to various physical production processes, such as level indication transmitter (LIT), flow indication transmitter (FIT), and differential pressure indicator transmitter (DPIT). Simultaneously, a discrete binarization layer was employed to handle the actuator values. Actuator values typically involve discrete control signals or commands sent to actuators to execute actions, such as opening and closing pump commands. Let $s_{k,i}$ and $a_{k,i}$ denote the values of the ith sensor and actuator, respectively, within the kth stage. After inputting a sensor value $s_{k,i}$, the continuous binarization layer checks whether $s_{k,i}$ satisfies the defined boundary conditions and constructs the corresponding one-dimensional binary tensor, denoted as $\bar{\mathbf{s}}_{\mathbf{k,i}}$

$$\bar{\mathbf{s}}_{\mathbf{k,i}} = [f(s_{k,i} < \mathrm{LL}), f(s_{k,i} > \mathrm{HH}), f(\mathrm{HH} \geq s_{k,i} \geq \mathrm{LL}), \ldots]. \tag{1}$$

LL and HH represent the lower and upper limits of the sensor used to monitor the state of the production materials within the container, respectively, and are determined by the operational requirements. For example, an LIT sensor monitors the liquid level in a raw water tank, and its values indicate the current level of the liquid. In addition to the range specified by the sensor, the boundary conditions can be determined based on the range of numerical fluctuations during a certain period of a known attack (e.g., values that frequently exceed or fall below a specific threshold). This method is grounded in real-world domain prior knowledge and diverges from random sampling.

After inputting the actuator value $a_{k,i}$, the discrete binarization layer assesses whether $a_{k,i}$ corresponds to the ON or OFF two distinct control states and outputs the corresponding binary encoding, denoted as $\bar{\mathbf{a}}_{\mathbf{k,i}}$

$$\bar{\mathbf{a}}_{\mathbf{k,i}} = [f(a_{k,i} = \mathrm{ON}), f(a_{k,i} = \mathrm{OFF})]. \tag{2}$$

When the input feature value satisfies the designated boundaries or states outlined in the conditions, the corresponding output is set as $f(\cdot) = 1$. Otherwise, $f(\cdot) = 0$. Let the binary state encoding for all devices within the kth stage be denoted as $\mathbf{h}^{0,k} = \bar{\mathbf{a}}_k \oplus \bar{\mathbf{s}}_k (k \in K)$, where $\bar{\mathbf{a}}_k$ and $\bar{\mathbf{s}}_k$ denote the concatenated binary encodings of all actuators and sensors at stage k, respectively. K represents the total number of physical processing stages in a specific ICS.

C. Logical Learning Module

The logical learning module automatically utilizes logical neural-symbolic representations to learn feature relationships based on binary encoding. The impacts stemming from attacks commonly manifest within a specific stage before propagating to other stages. Based on this observation of industrial processes, the module was partitioned into two parts. The initial logical layer is a single-stage logical layer that primarily concentrates on capturing interdevice relationships within the same stage. The subsequent logical layers are multistage logical layers designed to capture operational logic spanning diverse stages, which augments the single-stage logic. This serves as an interpretable constraint on the structure, making the learned logic rules intuitively responsive to control processes. In the following, we discuss the logical neural-symbolic representations used by each logical neuron.

1) Logical Neural-Symbolic Representation: Each logical layer consists of two distinct parts, namely a conjunction part and disjunction part, which correspond to the logical operators "AND" and "OR," respectively. Let $\mathbf{u}^l$ and $\mathbf{v}^l$ represent the conjunction and disjunction of the lth logical layer $\mathbf{h}^l$. Specifically, we define u_i^l as the ith node in $\mathbf{u}^l$ and v_i^l as the ith node in $\mathbf{v}^l$. The definitions of these two types of nodes are as follows:

$$u_i^l = \bigwedge_{W_{i,j}^{l,n}=1} \mathbf{h}^{l-1}, \quad v_i^l = \bigvee_{W_{i,j}^{l,n}=1} \mathbf{h}^{l-1}. \tag{3}$$

Here, $W_{i,j}^{l,n}$ represents the weight parameters between the logical layers, which are assigned discrete values of either zero or one, and n represents the number of times the weights are calculated. Fig. 3 illustrates this procedure. $W_{i,j}^{l,n} = 1$ indicates the existence of a logical connection between the current layer's node h_i^l and the previous layer's node h_j^{l-1}.

Considering the attack of intercepting PLC data packets and maliciously modifying pressure sensor values as an example, the conjunction part can detect inconsistencies between the actual production state and system operation. For instance, when the

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

TABLE II
TRUTH TABLE OF LOGICAL ACTIVATE FUNCTION

h	W	F_c
0	0	1
0	1	0
1	0	1
1	1	1

(a) Conjunction

h	W	F_s
0	0	0
0	1	0
1	0	0
1	1	1

(b) Disjunction

pipeline pressure increases, the system closes the valve intended for releasing gas, and this abnormal behavior can be identified. Meanwhile, the disjunction part captures various abnormal conditions. Whenever any sensor detects that the pipeline gas pressure exceeds the normal range, it triggers an alert. Therefore, a wide range of potential attacks can be effectively addressed by employing both conjunction and disjunction neural units.

2) Single-Stage Logical Learning Layer: This layer is primarily designed to acquire an understanding of the logic of a device within a particular physical processing stage. Consequently, this layer is partitioned into K submodules, and the input of each submodule, $\mathbf{h}^{l,k}$, is derived from the binary encoding of the devices in the corresponding stage. Although a logical neural-symbolic representation can articulate the decision process of a model in the form of rules, the binarization of its weight parameters renders the structure nondifferentiable, posing obstacles to the application of gradient-based optimization techniques [15], [16], [17]. Therefore, logical activation functions have been employed as an alternative approach

$$u_i^{l,k} = \prod_{n=1}^{m} F_c\left(\mathbf{h}^{0,k}, W_{i,j}^{l,n}\right)$$

$$v_i^{l,k} = 1 - \prod_{n=m+1}^{2m} \left(1 - F_s\left(\mathbf{h}^{0,k}, W_{i,j}^{l,n}\right)\right). \quad (l=1). \tag{4}$$

We allocated m conjunction and disjunction logic units for each device in the physical stage to extract the potential relationships among devices within the same stage. Parameter m defines the number of connections between each logical neuron and the units within the preceding layer (i.e., $m = 2$), which enables administrators to adjust the complexity of the rules according to the real scenario. F_c and F_s are Boolean functions, and h represents an element in the logical layer vector $\mathbf{h}$

$$F_c(h, W) = (1 - W \cdot (1 - h)), F_s(h, W) = W \cdot h. \tag{5}$$

Table II presents the truth tables corresponding to different weights and inputs. W can be interpreted as a logical feature selector. This ensures that, except for the node corresponding to $W = 1$, the remaining nodes exert no influence on the overall logic. Therefore, both activation functions satisfied the previously defined requirements for conjunctions and disjunctions.

To integrate logical layer weights into the aforementioned Boolean functions, it is imperative to map the continuous weight parameters π to discrete binary values of zero or one. The purpose of the weights is to select node h_j^{l-1} from among all the nodes within the preceding layer that exhibit the highest activation probabilities toward node h_i^l. The weights were calculated as follows:

$$W_{i,j}^{l,n} = \text{onehot}\left(\underset{j}{\text{argmax}}\left(\pi_{i,j}^{l,n}\right)\right). \tag{6}$$

Given that the argmax function is nondifferentiable, we use the Gumbel–Softmax estimator [18] as a differentiable approximation

$$W_{i,j}^{l,n} = \text{onehot}(\text{softmax}(\pi_{i,j}^{l,n} + g_j)/\tau). \tag{7}$$

Here, g_j is sampled from a Gumbel distribution (0,1), and τ represents temperature. To ensure diversity in feature selection, we started at a high temperature ($\tau = 1$) and annealed to a small temperature ($\tau = 0.0001$) for training. Subsequently, we concatenated all logical units stemming from different stages as the output of the current layer (i.e., $\mathbf{h}^1 = \mathbf{h}^{1,1} \oplus \cdots \oplus \mathbf{h}^{1,K}$).

3) Multistage Logical Learning Layers: Building upon device logic, the objective of the layers is to acquire operational logic spanning stages with the goal of encompassing the impacts of attacks across different processing stages comprehensively. Instead of distinguishing logical features belonging to different stages, the input of the multistage logical learning layers incorporates all the logical features from the previous layer for computation

$$u_i^l = \prod_{n=1}^{m} F_c\left(\mathbf{h}^{l-1}, W_{i,j}^{l,n}\right)$$

$$v_i^l = 1 - \prod_{n=m+1}^{2m} \left(1 - F_s\left(\mathbf{h}^{l-1}, W_{i,j}^{l,n}\right)\right). \quad (l > 1). \tag{8}$$

The computational procedures for F_c and F_s were the same as those described in (5). Furthermore, to handle instances of comparatively straightforward abnormal logic arising from attacks effectively, we incorporated residual connections. This enables the model to adaptively modulate the overall complexity of the rules for different types of attacks.

D. Prediction Module

From the logical-learning module, we obtained N_L new features, each corresponding to a distinct logical rule. By leveraging these features, we established an attack multiclassifier using a fully connected layer. To enhance the human comprehension of rules, we applied the softplus function to map the parameter weights into a nonnegative interval

$$\tilde{y}_i = \log(1 + \exp(\mathbf{w}_i)) \cdot \mathbf{h}^l. \tag{9}$$

Here, $\mathbf{w}_i$ represents the importance of different logical features for identifying the ith attack class. $\tilde{y}_i$ denotes the predictive probability that a sample belongs to the ith class. To make the final classification decision, we select the class with the highest probability as the predicted attack category.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

E. Training Objective

In ALRL, better model performance implies that more accurate rules can be extracted from the model. To further improve the classification performance of the proposed model, knowledge distillation [19] was employed to transfer knowledge between heterogeneous models.

The loss function of the model consists of two components: classification loss L_{hard} and distillation loss L_{dist}. L_{hard} measures the discrepancy between the model predictions $\tilde{y}_i$ and ground-truth labels c_i. L_{dist} quantifies the differences between the model predictions $\tilde{y}_i$ and the classification probabilities z_i of the teacher model, which are defined as follows:

$$L_{\text{hard}} = -\sum_{i}^{N} c_i \log(\text{softmax}(\tilde{y}_i))$$

$$L_{\text{dist}} = -\sum_{i}^{N} \text{softmax}(z_i/T)\log(\text{softmax}(\tilde{y}_i/T)). \tag{10}$$

The final objective function is defined as follows:

$$L_{\text{loss}} = T * T * \alpha L_{\text{dist}} + (1-\alpha)L_{\text{hard}}. \tag{11}$$

Considering the gradient scaling of $1/T^2$ in L_{dist}, it is necessary to multiply the distillation loss by T^2 to maintain a balance between the classification and distillation losses during the update process. α is a hyperparameter used to adjust the ratio between the two losses.

F. Explanation Module

The interpretation of ALRL is a progressive process that can be simplified by considering the binary and logical learning modules as feature learners. In both modules, each layer constructs new features based on information from the preceding layer and is guided by logical rules. Consequently, the final layer contains all the knowledge acquired by the model.

1) Constructing a Candidate Rule Set for Attacks: To extract the attack rules pertaining to a specific attack category c, we extract the fully connected weights of the cth row of the prediction module. By sorting the weight values in descending order and analyzing the corresponding features, we can define a threshold (th $= 10$) to filter and understand the most important rules. These rules and their corresponding weights w constitute the candidate rule set RS_c

$$\text{RS}_c = \{\text{rule}_1 : w_1, \text{rule}_2 : w_2, \ldots, \text{rule}_{\text{th}} : w_{\text{th}}\}. \tag{12}$$

Each rule in the candidate rule set consists of multiple conditions and logical operators ($\wedge$ and $\vee$). These conditions describe the devices and their decision boundaries based on the output of the binarization module.

2) Generating the Detection Rules Based on the Candidate Rule Set: To enhance readability, we simplify each rule in RS_c while preserving its original logical meaning. For example, if multiple conditions in a conjunction rule describe the same sensor, we maintain these conditions with a smaller decision boundary. To generate the required deterministic rules, we split the disjunctive rules into multiple logically independent conditions based on the distributive laws of Boolean algebra [20], to identify the satisfied condition for a given attack. Subsequently, we randomly selected n attack samples and represented each sample as x_n, where x_n included various device values. We then calculated the degree of matching for each independent condition as follows:

$$m_{\text{cond}^*} = \sum_{n} f(\text{cond}^*; x_n)/n. \tag{13}$$

Function f checks whether the values of input x_n satisfy the specific cond^*. If the condition is satisfied, the output is set as $f(\cdot) = 1$. We select the independent conditions with the highest attack matching rates (i.e., $m_{\text{cond}^*} > 0.9$) to construct the attack condition set $\{\text{cond}_1^*, \text{cond}_2^*, \ldots, \text{cond}_p^*\}$, which is then linked utilizing the logical AND operator. Consequently, deterministic rules R_c can be generated for this attack as follows:

$$R_c = \{\text{cond}_1^* \wedge \text{cond}_2^* \wedge \cdots \wedge \text{cond}_p^*\}. \tag{14}$$

As an example, consider a malicious attack to open the main water valve MV101 when the water tank LIT101 is full. The explanation module will output a series of candidate rules with high weights, involving conjunctions (e.g., $r_1 = c_1 : 871.254 \geq \text{LIT101} \geq 814.377 \wedge c_2 : 925.032 \geq \text{LIT101} \geq 575.032$) and disjunctions (e.g., $r_2 = c_3 : \text{MV101} = \text{ON} \vee c_4 : \text{LIT101} = 700$). To generate the determined rule, redundant rules were removed. $r_1 = c_1 \wedge c_2$ can be simplified as $r_1 = c_1$, for c_1 is a subset of c_2 in conjunction. Rules that were irrelevant to this type of attack were eliminated. $r_2 = c_3 \vee c_4$ can be simplified as $r_2 = c_3$, for the attack samples do not satisfy c_4 but c_3 in disjunction. Through these processes, we simplified the description of the abnormal rules. Finally, by connecting the remaining rules through conjunctions, we obtain a streamlined and effective set of deterministic rules $c_1 \wedge c_3$.

IV. Experiments and Evaluations

This section initially clarifies the datasets and experimental setup and then presents the comprehensive experimental results, aiming at answering the following questions.

Q1. Does the knowledge distillation of ALRL contribute positively to detection performance?

Q2. Can the rule-based interpretable ALRL model achieve detection accuracy comparable to that of well-performing intrusion detection models?

Q3. How are the decisions made by ALRL explained?

Q4. What are the characteristics of the detection rules learned by ALRL?

A. Datasets Description

We took two public industrial control datasets to conduct our experiment.

1) Secure Water Treatment (SWaT) [21]: This is a publicly available dataset from a water treatment test bed, which is a downscaled industrial replica of a water treatment plant. It has six stages and can produce five gallons of filtered water per minute. We selected 35 practical impact attacks to evaluate the

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

performance of the proposed model. Real-time data related to gthe production of the 25 sensors and 26 actuators were collected. This dataset includes various attacks targeting complex physical processes that are advantageous for evaluating IDSs.

2) Water Distribution Testbed (WADI) [22]: This is a publicly available dataset from a water resource distribution simulation system and is an extension of the SWaT dataset. There are three stages related to water distribution. The attacks encompassed 13 categories of cyberattacks, and real-time data were collected from 69 sensors and 54 actuators.

In both datasets, we used four attack scenarios to test ALRL's performance.

1) Single-stage single-point (SSSP): An SSSP attack focuses on exactly one point in the ICS.
2) Single-stage multipoint (SSMP): An SSMP attack focuses on two or more attack points in an ICS but only on one industrial process stage.
3) Multistage single-point (MSSP): An MSSP attack is similar to an SSMP attack, except that the SSMP attack is performed in multiple stages.
4) Multistage multipoint (MSMP): An MSMP attack is an SSMP attack that is performed in two or more stages of the ICS.

B. Experimental Methodology and Settings

1) Data Preprocessing: The SWaT dataset contains 4 499 919 raw data samples, while the WADI dataset contains 172 801 raw data samples. Abnormal samples are labeled with attack types based on publicly available attack manuals. Considering the nature of the target data sequences, we retained all attack samples and selected a set of samples occurring 1000 s before each attack type as normal samples to address imbalances in sample quantities. After downsampling, the SWaT dataset contains 47 044 samples, including 26 877 normal samples. The WADI dataset contains 20 947 samples, including 11 087 normal samples.

2) Baselines for Attack Classification: To demonstrate the validity of our method, we compared it with seven existing well-performed methods. *Rule-based methods*: The classification and regression tree (CART) [23] is known for its interpretability, making it suitable for understanding decision-making processes. However, the generated decision trees, particularly those with numerous continuous features, can become overly complex. Therefore, we employed cost-complexity pruning with pruning values of 0.003, 0.001, and 0.0, assessing the impact of different pruning thresholds on interpretability and performance. Higher values indicated simpler trees. A rule-based representation learner (RRL) [17] can automatically learn interpretable nonfuzzy rules for data representation and classification, which represents a state-of-the-art rule-based classification model.

ML methods: XGBoost [24] is a powerful gradient boosting tree algorithm that integrates multiple weak classifiers into a strong classifier. A piecewise linear neural network (PLNN) [25] represents an MLP in a mathematically equivalent linear form that adopts piecewise linear activation functions, such as rectified linear unit (ReLU), which can offer explanatory insights into input samples. These complex ML methods generally achieve better detection performance than interpretable rule-based methods but compromise interpretability to a great extent.

DL methods: The self-attention and intersample attention transformer (SAINT) [26] proposed a novel enhanced embedding method based on a deep transformer network that demonstrated strong performance in processing data samples with a mixture of continuous, categorical, and ordinal values. 1-D convolutional neural networks (1D-CNN) [10] are effective cyberattack detection mechanisms in ICS. They achieve optimal performance while maintaining generality, simplicity, and a small footprint. Logic understanding IDS (LU-IDS) [13] is a recently proposed intrusion detection method. A specific DL module for the ICS was designed based on a residual network. Owing to its capability of capturing rich feature representations from data during training, it can serve as a powerful performance benchmark. Therefore, we utilized this particular residual network as a teacher model to transfer its learned representation to our model, which can potentially enhance the learning process and improve performance.

3) Parameter Settings: Within the logical-learning module, we set $L = 2$. For the single-stage logic learning layer, we configure the number of logic units based on the number of sensors and actuators at each distinct physical stage of the dataset. In the SWaT dataset, in which the number of devices designed for each physical stage was approximately equal, we set the number of disjunctive and conjunctive logic units to 24. In the WADI dataset, as the secondary grid stage encompasses multiple water tanks, requiring a larger set of physical devices for monitoring than the other two stages, we set the number of logic units to {24, 72, 24} to accommodate the differences. For the multistage logic learning layers, considering the model complexity and computational efficiency, we uniformly set the number of conjunction and disjunction units to 64. We utilized Adam for the training process with a mini-batch size of 32, which was selected for its efficient handling of sparse gradients [27]. We adopted a learning rate decay strategy in which the initial learning rate started at 0.01 and linearly decreased to 0.001. It provides a smoother transition in the optimization landscape, with fast convergence in the early stage and an optimal solution in the late stage. The training was conducted over 50 epochs.

We used the scikit-learn default parameters for CART and XGBoost. Ten estimators were used for the ensemble models. For RLL and PLNN, we set the optimizer selection, learning rate, decay rate, batch size, model depth, and width to be the same as those for ALRL. For the 1D-CNN, we constructed a model comprising five convolutional and three linear layers. These convolutional layers conduct one-dimensional convolutions with kernel sizes decreasing from five to one and a stride of one. To perform attack classification, we replaced the final fully connected layer based on the number of attack categories. For SAINT, we set the number of stages to 1, the number of attention heads in each attention layer to 4, the embedding size to 32, and the dropout rate to 0.8. In the data preprocessing of the 1D-CNN and SAINT, the features were normalized to the (0,1) scale. For the LU-IDS, we encoded the features using IEEE754 and input them into a residual network composed of four residual

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

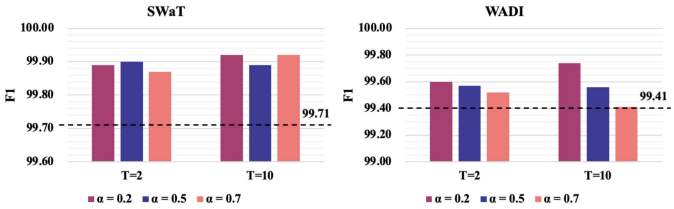


Fig. 4. Attack classification performances using different values of α and T. The black dashed line shows the performance when $\alpha = 0$.

blocks. Each residual block consists of two convolutional layers with a kernel size of 3×1.

4) Evaluation Metrics: The experiments were conducted on a Linux server equipped with a GeForce RTX 3070 GPU. We adopted accuracy, precision, recall, and $F1$ (macro) as attack classification performance metrics because the datasets were imbalanced. We adopted a five-fold cross-validation to evaluate the classification performance more fairly, with 80% of the data used for model training and the remaining 20% used for validation. The calculation details are as follows:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}, \text{Precision} = \frac{\text{TP}}{\text{TP+FP}}$$
$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, F1 = \frac{2 * \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (15)$$

TP is a normal sample classified as a normal sample, and TN is an attack sample classified as an attack sample. FP indicates an attack sample classified as normal, and FN indicates a normal sample classified as an attack.

In addition, we calculated the false alarm rates (FAR) for evaluating the performance of intrusion detection with the rules on the two public datasets as follows:

$$\text{FAR} = \frac{\text{FN}}{\text{FN} + \text{TP}}. \quad (16)$$

In both datasets, each sample encompassed all the device values per second. Therefore, for a more comprehensive evaluation of the FAR, we selected all normal samples from the above-mentioned datasets and calculated the ratio of normal samples mislabeled as attacks to the total number of normal samples.

C. Distillation Temperature Study (Q1)

We presented performance comparisons for ALRL with different distillation temperatures T and distillation loss proportions α. The results are shown in Fig. 4. The black dashed line serves as a baseline, representing the performance of ALRL when it does not utilize knowledge from the teacher model during the training process. The results demonstrated that the performance of ALRL with distillation loss surpassed that of the baseline, highlighting the contribution of knowledge from the teacher model. Although the neural network alone has already achieved a good performance, further enhancements can be achieved by choosing different temperatures and ratios. Therefore, we employed the set of parameters yielding the highest model performance for subsequent experiments on both datasets, namely $\alpha = 0.2$ and $T = 10$.

TABLE III
ATTACK CLASSIFICATION PERFORMANCE ON THE SWAT DATASET

Method	Accuracy	Precision	Recall	F1(macro)	Parameters	Time
CART(0.003)	0.9579	0.9560	0.8864	0.9089	-	-
CART(0.001)	0.9783	0.9835	0.9532	0.9646	-	-
CART(0.0)	0.9935	0.9910	0.9922	0.9905	-	-
RRL	0.9846	0.9690	0.9799	0.9725	0.22M	17
PLNN	0.9995	0.9967	0.9992	0.9979	0.24M	3
XGBoost	0.9970	0.9995	0.9913	0.9953	-	-
SAINT	0.9943	0.9880	0.9948	0.9911	47.13M	90
1D-CNN	0.9929	0.9866	0.9927	0.9883	1.45M	19
LU-IDS	0.9993	0.9987	0.9995	0.9991	3.81M	65
ALRL	0.9992	0.9996	0.9987	**0.9992**	**0.09M**	**11**

The best results are in bold.

TABLE IV
ATTACK CLASSIFICATION PERFORMANCE ON THE WADI DATASET

Method	Accuracy	Precision	Recall	F1(macro)	Parameters	Time
CART(0.003)	0.9823	0.9849	0.9695	0.9768	-	-
CART(0.001)	0.9895	0.9856	0.9871	0.9861	-	-
CART(0.0)	0.9926	0.9877	0.9922	0.9903	-	-
RRL	0.9883	0.9725	0.9769	0.9742	0.40M	12
PLNN	0.9976	0.9962	0.9945	0.9952	0.29M	2
XGBoost	0.9976	0.9933	0.9972	0.9952	-	-
SAINT	0.9888	0.9668	0.9915	0.9757	66.28M	60
1D-CNN	0.9916	0.9911	0.9790	0.9845	1.71M	10
LU-IDS	0.9974	0.9955	0.9991	0.9973	3.82M	55
ALRL	0.9981	0.9969	0.9979	**0.9974**	**0.24M**	**7**

The best results are in bold.

D. Model Performances for Attack Classification (Q2)

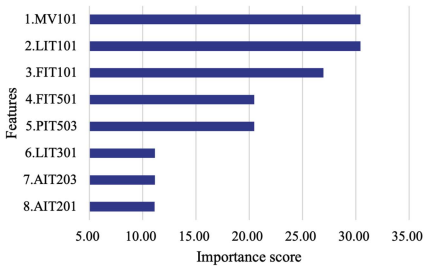
Tables III and IV present the average performance measures of ALRL for attack classification, following a five-fold cross-validation of the SWaT and WADI datasets. The results indicate that, compared to other interpretable rule-based methods, ALRL achieves outstanding attack classification performance across both datasets. Although complex ML/DL approaches exhibit a performance comparable to that of ALRL in the domain of attack classification, their limitations in terms of model interpretability impede a deeper understanding of the underlying attack behaviors.

Another advantage of ALRL is its low training cost. In the same training environment, we use the number of learnable parameters and training time per epoch as metrics to show the model complexity. Since CART and XGBoost employ heuristic learning methods, they do not have learnable parameters and were, thus, excluded from the comparison. The results show that ALRL requires fewer model parameters and less training time to achieve an excellent detection performance, making it a promising choice for real-world use in settings with limited computational resources.

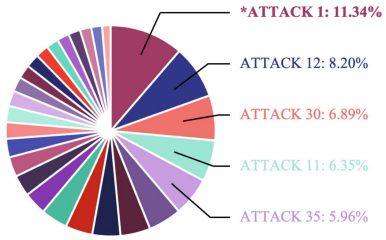
E. Explaining the Decisions of ALRL (Q3)

Before obtaining specific detection rules, we can gain intuitive insights into attacks from a global perspective by analyzing the weights and semantic information associated with the features

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Reason(*condition*)	Effect
LIT101 = 867.643 (*871.254 >= LIT101 >= 814.377*) MV101 = ON (*MV101 = ON*)	+3.19%
FIT101 = 2.659 (*2.671 >= FIT101 >= 0*) LIT101 = 867.643 (*925.032 >= LIT101 >= 575.032*)	+2.83%
PIT503 = 188.541 (*189.519 >= PIT503 >= 188.525*) FIT501 = 1.723 (*1.732 >= FIT501 >= 1.722*)	+2.14%
FIT101 = 2.65 (*2.671 >= FIT101 >= 0*)	+1.69%
FIT101 = 2.65 (*2.696 >= FIT101 >= 2.416*) MV101 = ON (*MV101 = ON*)	+1.47%
Final prediction	**11.34%**

(c)

Fig. 5. Explaining a sample of Attack 1 on the SWaT dataset. (a) A summary plot of top important features in predicting attack class 1. (b) A pie chart shows the predict probability based on the top 5 important features, label marked with asterisks denote the decision outcomes. (c) A contribution table illustrates how the important features of the sample effect the prediction of attack class 1.

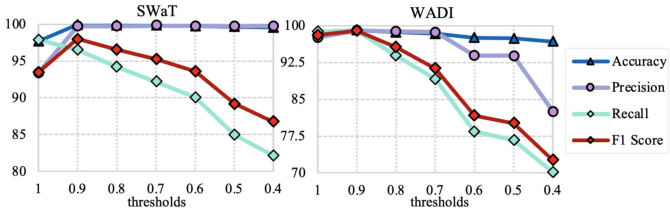


Fig. 6. Performance degradation curve of the rules at different thresholds.

in the explanation module. This can significantly aid administrators in quickly comprehending anomalies within an ICS. Fig. 5 presents an example of the detailed explanatory information provided by the ALRL in a specific attack scenario on the SWaT dataset. Fig. 5(a), we observe that both actuator MV101 and sensor LIT101 play crucial roles in the model determination. Fig. 5(b) and 5(c) showcases the top five important features, allowing us to roughly classify the attack samples. When the sample's feature values satisfy the conditions expressed by the semantic information, the prediction is positively influenced by 11.34%, with Attack 1 having the highest probability. This is consistent with realistic results.

F. Rule Performances for Intrusion Detection (Q4)

We analyzed the effectiveness and simplicity of the deterministic intrusion detection rules generated by ALRL as follows.

1) Parameter Sensitivity: We studied the importance of the model parameters by varying the thresholds to show how the attack matching rates affect the quality of the generated detection rules. Fig. 6 presents the performance degradation curves of the rules under different thresholds. We observe that the rules achieved optimal performance at a threshold of 0.9. This is attributed to the fact that excessively low thresholds result in underfitting, failing to adequately capture the characteristics of attacks, whereas excessively high thresholds lead to overfitting, in which the generated rules make overly stringent assumptions about attacks and fail to effectively identify all attack samples. These findings inspired us to achieve better results by selecting an appropriate threshold based on the scenario, thereby ensuring the generalization of rules while maintaining basic detection accuracy.

TABLE V
GENERATED RULE ATTACK DETECTION PERFORMANCE FOR ALRL ON THE SWAT DATASET†

ID	Mentioned sensors and actuators in rules (ordered by weights)	F1
1	LIT101, **MV101**, FIT101, PIT503, FIT501, AIT201	0.993
2	**P102**, LIT301, AIT504	0.982
3	AIT202, AIT203, AIT201, P203, AIT402, LIT301	0.975
4	AIT202, AIT203, LIT401, LIT101, AIT503, LIT301	0.997
5	P203, AIT201, **AIT202**, P204, LIT101, P101, LIT301	0.983
6	P203, AIT402, PIT503, LIT101, FIT101	0.942
7	**DPIT301**, AIT501, AIT201	0.949
8	**FIT401**, AIT202, LIT301, AIT203, LIT101, FIT101	0.972
9	**FIT401**, UV401, AIT201, FIT201	0.984
10	AIT202, AIT203, LIT101, LIT301, LIT401, AIT504	1.000
11	LIT101, AIT503, AIT203, AIT501, AIT502, LIT301	0.989
12	**LIT301**, AIT501, AIT504, AIT402, LIT401	0.972
13	AIT503, AIT504, LIT101, P101, AIT501, AIT202, AIT201	0.930
14	PIT501, **AIT504**, LIT301, AIT501, PIT502	0.996
15	FIT503, **AIT504**, LIT301, LIT401, PIT502	0.894
16	**LIT101**, FIT101, AIT203	0.997
17	AIT202, **UV401**, AIT503, FIT601	0.999
18	**DPIT301**, AIT203, AIT201, LIT301, P302, AIT501	1.000
19	AIT201, **P203**, LIT401, AIT402, LIT101, FIT101	0.941
20	**LIT401**, MV101, AIT201, P203, AIT202	0.975
21	P102, **LIT301**	0.999
22	**LIT401**, LIT101	0.993
23	LIT301, AIT201, AIT503, AIT501, AIT504, FIT502	0.999
24	P102, **LIT101**	0.997
25	**LIT401**, PIT501	1.000
26	AIT501, **LIT301**, AIT502, FIT401, DPIT301, LIT401	0.980
27	**LIT101**, MV101, AIT203	0.994
28	P102, LIT301, **P101**	1.000
29	LIT401, PIT501, AIT504, AIT201, P203, AIT501, FIT201, PIT503, **P101**	0.997
30	LIT301, AIT203, AIT501, PIT501, LIT401, PIT503, AIT402	1.000
31	LIT301, LIT401, **LIT101**	0.928
32	**AIT402**, AIT504, FIT502, AIT501, LIT301, MV101	0.993
33	**AIT502**, FIT501, LIT301, P201, AIT203, FIT601	0.972
34	**FIT401**, UV401, P201, LIT301, FIT601, LIT401	0.974
35	AIT503, AIT501, **FIT301**, AIT201, AIT203, LIT101, P101	0.990

† Bold text signifies the labeled attack point as indicated in the attack manual.

TABLE VI
GENERATED RULE ATTACK DETECTION PERFORMANCE FOR ALRL ON THE WADI DATASET

ID	Mentioned sensors and actuators in rules (ordered by weights)	F1
1	3_AIT_005_PV, 2_LT_001_PV, 1_MV_001_STATUS, 1_P_003_STATUS, 1_AIT_001_PV	0.992
2	1_AIT_004_PV, 1_AIT_005_PV, 2_FIC_301_SP, 3_FIT_001_PV, 2A_AIT_004_PV, 2B_AIT_001_PV	0.986
3	2_PIT_003_PV, 2A_AIT_004_PV, 2_PIC_003_PV, 2_LT_002_PV	0.973
4	2_MCV_501_CO, 1_AIT_004_PV, 2_PIT_001_PV, 1_AIT_001_PV, 2_FIC_301_SP, 2B_AIT_001_PV, 3_FIT_001_PV, 3_AIT_003_PV, 2_DPIT_001_PV	1.000
5	1_AIT_004_PV, 1_AIT_001_PV, 3_LT_001_PV, 3_FIT_001_PV, 2_FQ_101_PV	0.995
6	2B_AIT_004_PV, 2_LT_002_PV, 2A_AIT_004_PV	0.963
7	1_AIT_001_PV, 1_AIT_004_PV, 1_FIT_001_PV, 1_LT_001_PV, 2_MCV_007_CO	0.981
8	2_LT_002_PV, 2_P_003_SPEED, 2_PIT_003_PV, 2_FIT_003_PV	0.999
9	2_PIT_001_PV, 2_FIC_201_CO, 2_MCV_007_CO, 2_FQ_101_PV, 2_MCV_501_CO, 2_FIT_002_PV, 1_AIT_003_PV, 1_AIT_004_PV, 2_MCV_301_CO, 1_AIT_002_PV	0.995
10	2_FQ_501_PV, 2_P_003_SPEED, 2_PIT_002_PV, 2_MCV_007_CO, 2_MCV_301_CO	0.991
11	1_AIT_004_PV, 2B_AIT_002_PV, 3_FIT_001_PV, 2A_AIT_004_PV, 2_PIT_001_PV, 3_LT_001_PV	0.996
12	1_AIT_004_PV, 1_FIT_001_PV, 1_P_001_STATUS, 2_FQ_401_PV, 2_MCV_101_CO, 2_FIC_601_CO, 2_PIC_003_CO	0.997
13	2_MCV_301_CO, 2_FIC_101_CO, 2_FIC_401_CO, 2B_AIT_004_PV, 1_LT_001_PV, 2_PIC_003_PV	0.999

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PERFORMANCES OF GENERATED RULES IN INTRUSION DETECTION TASKS ON TWO DATASETS

	SWaT			WADI		
Scenarios	Attacks	$F1$	FAR(%)	Attacks	$F1$	FAR(%)
SSSP	23	0.978	0.0108	7	0.991	0.0021
SSMP	5	0.967	0.3954	2	0.997	0.0009
MSSP	4	0.989	0.0000	2	0.980	0.3498
MSMP	3	0.998	0.0010	2	0.986	0.2590
Total	35	0.980	0.0637	13	0.990	0.0949

2) Individual Rules Detection Performance: Tables V and VI present the rules generated from the ALRL and the corresponding detection performance. On the SWaT dataset, our rules successfully detected all 35 attack types with an accuracy exceeding 90%, and the attack points were identified early within the rules. For the WADI dataset, the rules achieved a detection accuracy of at least 95% for each of the 13 attack categories tested. This indicates that the ALRL-extracted rules effectively captured the core abnormal logic for each attack type.

3) Comparison of Rule Ensemble Effects: Table VII presents the rule ensemble performance for intrusion detection using the rules generated by ALRL on the SWaT and WADI datasets. We evaluated our rules against the four attack scenarios provided in the abovementioned two datasets and calculated the F1-score and FAR for each attack scenario, where samples corresponding to specific attacks were marked as anomalies and all others were considered normal. Our rules consistently exhibited high $F1$-scores across various attack scenarios. In addition, our rules resulted in only a small number of false positives: 0.0637% on the SWaT dataset and 0.0949% on the WADI dataset. This result proves that despite the abundance of samples, these rules do not lead to an excessively high FAR.

To further emphasize the strong interpretability of the rules, we compared ALRL with two state-of-the-art rule-based methods: CART [23] and LU-IDS [13]. We adopted the rule set size, min and max rule lengths, average rule length, information entropy H_s, and $F1$-score as the metrics to evaluate the rules. H_s represents the amount of useful information contained in each rule condition and is calculated as follows:

$$H_s = \sum_{i=1}^{n} p_i I_e = -\sum_{i=1}^{n} p_i \log_2 s_i \tag{17}$$

where $p_i = 1/n$, and n denotes the number of conditions in the rule. Accuracy was adopted as a measure of self-information s_i in this study. Under extreme conditions, when a certain condition within a rule does not assist in distinguishing between attacks, the accuracy of the rule becomes zero, leading to infinite entropy.

Table VIII presents the comparison results with rule-based methods. We observed that ALRL had the minimum number of rules in the rule set, and the rules had shorter lengths than those generated by other rule-based methods for both complex and simple attack scenarios within both datasets. Notably, while maintaining the same level of detection accuracy, ALRL's average rule length on the SWaT dataset was significantly reduced by 44.95% compared with that of LU-IDS. Similarly, on the WADI dataset, the average rule length decreased by 16.42% compared with that of the unpruned CART. In addition, the ALRL consistently demonstrated remarkably low entropy values across both datasets, signifying the distinct importance of each condition within the rules. Overall, our generated rules contain highly effective information that surpasses that of domain experts alone. The combination of rule effectiveness and simplicity in ALRL makes it an effective tool for industrial maintainers to identify and respond to attacks on ICSs.

4) Case Study: Based on the interrelationships between logical conditions, we can roughly classify the rules into three types. We provided a detailed description of the characteristics of each type.

a) Directly associated anomalous logic: **RULE1** : $871.254 \geq \text{LIT101} \geq 814.377 \wedge \text{MV101} = \text{ON} \wedge 2.671 \geq \text{FIT101} \geq 0 \wedge 189.519 \geq \text{PIT503} \geq 188.525 \wedge 1.732 \geq \text{FIT501} \geq 1.722 \wedge 265.188 \geq \text{AIT201} \geq 262.144$. The condition $\text{LIT101} \geq 814.377$ indicates that the original water tank storage value exceeded the maximum boundary HH (where HH = 800 in SWaT). In this scenario, the attacker maliciously opens the valve controlling the inflow of raw water MV101 = ON, resulting in a water tank overflow. Consequently, the state of the actuator directly affects the sensor value.

RULE34 : $\text{FIT401} = 0 \wedge \text{UV401} = \text{OFF} \wedge \text{P201} = \text{ON} \wedge 834.363 \geq \text{LIT301} \geq 580.363 \wedge \text{FIT601} = 0.0 \wedge 919.072 \geq \text{LIT401} \geq 775.109$. The conditions $\text{FIT401} = 0$ and $\text{UV401} = \text{OFF}$ indicate that, under certain circumstances, the attacker maliciously sets the value of FIT401 to zero, which triggers the closure of UV401. Consequently, the water flow to the reverse osmosis system decreases significantly, ultimately causing damage to subsequent components of the water treatment system. Therefore, the sensor values can influence the switching states of the actuators.

Overall, this rule demonstrates that the ALRL is capable of learning and extracting direct and effective relationships between sensors and actuators.

b) Abnormal logic influence among devices within the same stage: **RULE5** : $\text{P203} = \text{OFF} \wedge 263.522 \geq \text{AIT201} \geq 263.298 \wedge \text{AIT202} = 6 \wedge \text{P204} = \text{OFF} \wedge 523.632 \geq \text{LIT101} \geq 498.629 \wedge \text{P101} = \text{ON} \wedge 946.552 \geq \text{LIT301} \geq 854.509$. The majority of Rule 5 consists of devices within the same stage, including the chemical composition measurement instruments AIT201 and AIT202, as well as the control valves P203 and P204 for introducing chemical additives. The attacker forcibly set AIT202, which is used to measure chemical additives, to six. This action led to an erroneous assessment of the current water quality by the system, resulting in the improper closure of control valve P203 for chemical additive input. Ultimately, this negatively affected the water quality following sewage treatment. This indicates that ALRL can successfully learn and analyze the underlying logical relationships between chemical reagent measurement instruments and control devices within the same process stage.

c) Cross-stage abnormal operational logic: **RULE21** : $\text{P102} = \text{ON} \wedge \text{LIT301} = 801$. Despite consisting of only two

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
COMPARISON RESULTS WITH RULE-BASED METHODS ON TWO DATASETS†

Method	SWaT						WADI					
	SIZE	MIN	MAX	AVG	H_s	$F1$	SIZE	MIN	MAX	AVG	H_s	$F1$
CART (0.0)	108	3	22	11.874	1.156	**0.990**	51	3.2	15.3	6.902	1.182	**0.990**
CART (0.001)	55	3	21	11.335	1.175	0.965	38	3.2	12.8	6.501	1.213	0.986
CART (0.003)	43	3	22	11.033	1.180	0.909	32	**3**	**9.1**	6.251	1.229	0.977
LU-IDS (rules)	**35**	5	14	8.886	1.366	**0.990**	**13**	8	21	12.000	1.354	0.978
ALRL (rules)	**35**	**2**	**8**	**4.971**	**0.536**	0.980	**13**	**3**	10	**5.769**	**0.449**	**0.990**

† The best results are in bold and the second-best results are underlined.

conditions, this clearly reflects the existing operational logic between different stages. The attacker opened P102 while P101 was open, leading to increased water flow in the subsequent stage. Because the second stage was intended solely for the addition of chemical additives, excess water flowed into the storage tank of the third stage. Consequently, the water in storage tank LIT301 overflowed, surpassing the upper boundary HH (where HH = 800 in SWaT). Such multistage abnormal logic rules are valuable for administrators because they facilitate the rapid comprehension of attack points and their corresponding interconnected responses, even when administrators lack knowledge of the internal operational logic within a facility.

G. Discussion on the Challenges of Security Monitoring

Although the concise and effective rules generated by ALRL strengthen the security of ICSs, some challenges remain in their deployment in practical security monitoring.

1) *Online learning*: ALRL's training methodology involves the offline training of static data. However, real-world systems operate in dynamic environments where new threats continuously emerge.
2) *Rule updates*: In response to emerging threats, the timely incorporation of new rules or modifications to existing rules is necessary. However, the updating process may disrupt the performance of the existing rules.

Therefore, in future work, we will consider adopting incremental learning during the training process to endow the model with the ability of dynamic learning, which will augment the knowledge of the existing model using new data samples. Moreover, integrating security analyst feedback into the rule update loop can ensure the real-time efficacy of the rule set.

V. CONCLUSION

This article presents a novel intrusion detection method for ICSs called ALRL, which can accurately identify multiple types of attacks and generate the corresponding effective and human-readable detection rules. This is achieved by designing specific neural networks and knowledge distillation for attack classification, coupled with the extraction of anomalous control logic rules using logical neural-symbolic representation. The hierarchical logical learning model structure and residual connections resulted in the formation of simpler rules. Extensive experiments on two public datasets demonstrate that, compared to rule-learning methods, ALRL can automatically learn complex relationships between industrial control data features, exhibit outstanding classification performance, and be capable of generating effective and concise rules. Compared to DL methods, ALRL exhibits high interpretability owing to its structural transparency and low complexity, while maintaining comparable performance. In summary, ALRL emerges as a promising solution for enhancing the security of ICSs.

REFERENCES

[1] D. Bhamare, M. Zolanvari, A. Erbad, R. Jain, K. Khan, and N. Meskin, "Cybersecurity for industrial control systems: A survey," in *Comput. Secur.*, vol. 89, 2020, Art. no. 101677.
[2] J. Tidy, "European oil facilities hit by cyber-attacks," Feb., 2022. [Online]. Available: https://www.bbc.com/news/technology-60250956
[3] A. Jones, Z. Kong, and C. Belta, "Anomaly detection in cyber-physical systems: A formal methods approach," in *Proc. IEEE 53rd Conf. Decis. Control*, 2014, pp. 848–853.
[4] S. Adepu and A. Mathur, "Distributed attack detection in a water treatment plant: Method and case study," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 1, pp. 86–99, Jan./Feb., 2021.
[5] T. K. Das, S. Adepu, and J. Zhou, "Anomaly detection in industrial control systems using logical analysis of data," *Comput. Secur.*, vol. 96, 2020, Art. no. 101935.
[6] A. Ahmim, L. Maglaras, M. A. Ferrag, M. Derdour, and H. Janicke, "A novel hierarchical intrusion detection system based on decision tree and rules-based models," in *Proc. IEEE 15th Int. Conf. Distrib. Comput. Sensor Syst.*, 2019, pp. 228–233.
[7] F. Zhang, H. A. D. E. Kodituwakku, J. W. Hines, and J. Coble, "Multilayer data-driven cyber-attack detection system for industrial control systems based on network, system, and process data," *IEEE Trans. Ind. Inform.*, vol. 15, no. 7, pp. 4362–4369, Jul. 2019.
[8] T.-T.-H. Le, Y. E. Oktian, and H. Kim, "Xgboost for imbalanced multiclass classification-based industrial Internet of Things intrusion detection systems," *Sustainability*, vol. 14, no. 14, 2022, Art. no. 8707.
[9] Y. Fang, Y. Yao, X. Lin, J. Wang, and H. Zhai, "A feature selection based on genetic algorithm for intrusion detection of industrial control systems," *Comput. Secur.*, vol. 139, 2024, Art. no. 03675.
[10] M. Kravchik and A. Shabtai, "Efficient cyber attack detection in industrial control systems using lightweight neural networks and PCA," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2179–2197, Apr. 2021.
[11] H. T. Truong et al., "Light-weight federated learning-based anomaly detection for time-series data in industrial control systems," *Comput. Ind.*, vol. 140, 2022, Art. no. 103692.
[12] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 305–316.
[13] M. Sun, Y. Lai, Y. Wang, J. Liu, B. Mao, and H. Gu, "Intrusion detection system based on in-depth understandings of industrial control logic," *IEEE Trans. Ind. Inform.*, vol. 19, no. 3, pp. 2295–2306, Mar. 2022.
[14] S. Bahadoripoura, H. Karimipoura, A. N. Jahromia, and A. Islama, "An explainable multi-modal model for advanced cyber-attack detection in industrial control systems," *Internet Things*, vol. 25, 2024, Art. no. 101092.
[15] H. Qin, R. Gong, X. Liu, X. Bai, J. Song, and N. Sebe, "Binary neural networks: A survey," *Pattern Recognit.*, vol. 105, 2020, Art. no. 107281.
[16] P. Sen, B. W. de Carvalho, R. Riegel, and A. Gray, "Neuro-symbolic inductive logic programming with logical neural networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, 2022, pp. 8212–8219.
[17] Z. Wang, W. Zhang, N. Liu, and J. Wang, "Learning interpretable rules for scalable data representation and classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 2, pp. 1121–1133, 2024.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.

[18] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *Proc. 5th Int. Conf. Learn. Representations*, 2017, pp. 1–12. [Online]. Available: https://openreview.net/forum?id=rkE3y85ee
[19] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[20] P. Halmos and S. Givant, *Introduction to Boolean Algebras*. Berlin, Germany: Springer, 2009.
[21] A. P. Mathur and N. O. Tippenhauer, "SWAT: A water treatment testbed for research and training on ICS security," in *Proc. IEEE Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2016, pp. 31–36.
[22] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: A water distribution testbed for research in the design of secure cyber physical systems," in *Proc. 3rd Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2017, pp. 25–28.
[23] L. Breiman, *Classification and Regression Trees*. London, U.K.: Routledge, 2017.
[24] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 785–794.
[25] L. Chu, X. Hu, J. Hu, L. Wang, and J. Pei, "Exact and consistent interpretation for piecewise linear neural networks: A closed form solution," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1244–1253.
[26] G. Somepalli, A. Schwarzschild, M. Goldblum, C. B. Bruss, and T. Goldstein, "SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training," in *Proc. NeurIPS 1st Table Representation Workshop*, 2022, pp. 1–18. [Online]. Available: https://openreview.net/forum?id=FiyUTAy4sB8
[27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Representations*, pp. 1–15, 2015. [Online]. Available: http://arxiv.org/abs/1412.6980



Xinyu Xu received the B.E. degree in computer science and technology in 2022 from the Beijing University of Technology, Beijing, China, where he is currently working toward the M.S. degree in information security.

His research interests include intrusion detection, deep learning, and model interpretation.



Yingxu Lai (Member, IEEE) received the Ph.D. degree in computer science and technology from the Chinese Academy of Sciences, Beijing, China, in 2003.

She joined the College of Computer Science, Beijing University of Technology, Beijing, China, in 2003, and is currently a Full Professor. She was a Visiting Scholar with Arizona State University, Tempe, AZ, USA, from 2013 to 2014. She has had more than 70 papers published in various international journals and conferences. Her research interest covers cloud computing, network security, edge computing, and trusted computing.



Xiao Zhang received the B.E. degree in information security in 2022 from the Beijing University of Technology, Beijing, China, where she is currently working toward the Ph.D. degree in computer science and technology with the Institute of Information Security.

Her research focuses on knowledge graph and industrial control system security.



Xinrui Dong received the B.E. degree in information security in 2022 from the Beijing University of Technology, Beijing, China, where she is currently working toward the M.S. degree in information security.

Her research interests include industrial control systems security and intrusion detection. She also explored the field of deep learning and few-show incremental learning.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:08:02 UTC from IEEE Xplore. Restrictions apply.